Mês 8 — Demais Corpora + Custo em CPU (abr/2027)

Guia de Estudo - Projeto PIBITI: Detecção de Deepfakes de Áudio com WPT

Bolsista PIBITI

Abril de 2027

Sumário

1 O que o plano exige **2**

2 Incorporar novos corpora **3**
2.1 Template para novo corpus 3
2.2 Padronização de Labels e Metadados 3

3 Medir custo em CPU **5**
3.1 Usando `time.time()` 5
3.2 Medindo Leitura vs. Decomposição 5
3.3 Acompanhamento de Memória (`tracemalloc`) 6

4 Profiling detalhado **7**
4.1 Média de Múltiplas Execuções 7
4.2 Custo de Inferência do Modelo 7

5 Tabela de Custos **9**

6 Ajuste de escopo **10**

7 Pipeline de processamento em lote **11**

8 Brincadeiras: Otimizando o Tempo **12**
8.1 Brincadeira 1: Soma de Energia com For Loop vs NumPy 12
8.2 Brincadeira 2: `timeit` 12
8.3 Brincadeira 3: Profiling com cProfile 12
8.4 Brincadeira 4: Garbage Collection 13

9 Checklist **14**

10 Desafio final **15**

Capítulo 1

O que o plano exige

Neste oitavo mês, o projeto avança de experimentos restritos e controlados para uma avaliação de larga escala, medindo a real viabilidade das nossas propostas. Os objetivos centrais delineados pelo plano são:

1. **Estender aos demais *corpora*:** Até agora focamos no ASVspoof, mas é fundamental incluir o FakeBrAccent e o Fake Voices para avaliar a robustez entre bases distintas (*cross-corpus*).
2. **Medir e registrar o custo computacional:** Todo o processamento (extração de características, treinamento e inferência) precisa ter seu tempo de CPU e consumo de memória metrificados rigorosamente.
3. **Ajuste de escopo:** Conforme as medições revelarem altos custos computacionais, faremos uma expansão incremental. Só incluiremos mais combinações (Wavelet + Nível) se o orçamento de CPU permitir.

Marco / Entrega

Expansão da cobertura para demais *corpora* + Custo em CPU metrificado e registrado de maneira formal.

Capítulo 2

Incorporar novos corpora

Expandir o pipeline para múltiplos *corpora* exige padronização e código organizado. Ao lidar com FakeBrAccent e Fake Voices, os formatos podem variar, taxas de amostragem podem ser distintas, e até a estrutura de diretórios será diferente.

2.1 Template para novo corpus

O primeiro passo é abstrair a leitura de arquivos e anotações para um formato unificado.

```python
import os
import glob

def get_audio_files(corpus_path, extension="*.wav"):
    search_pattern = os.path.join(corpus_path, "**", extension)
    files = glob.glob(search_pattern, recursive=True)
    return files

corpus_fakebr_path = "./data/FakeBrAccent"
fakebr_files = get_audio_files(corpus_fakebr_path)
print(f"Total de audios no FakeBrAccent: {len(fakebr_files)}")
```

```python
corpus_fakevoices_path = "./data/FakeVoices"
fakevoices_files = get_audio_files(corpus_fakevoices_path)
print(f"Total de audios no FakeVoices: {len(fakevoices_files)}")
```

2.2 Padronização de Labels e Metadados

Cada corpus apresenta seu próprio formato de metadados. Precisamos de uma função que converta todos para a nomenclatura padrão (`bonafide` e `spoof`).

```python
def parse_fakebr_labels(metadata_path):
    labels = {}
    with open(metadata_path, 'r') as f:
        for line in f:
            parts = line.strip().split()
            # Assumindo formato: id_audio label
            if len(parts) >= 2:
```

```
                audio_id, label = parts[0], parts[1]
                # Padronizar
                if label.lower() in ['real', 'bonafide']:
                    labels[audio_id] = 'bonafide'
                else:
                    labels[audio_id] = 'spoof'
    return labels
```

```
def get_corpus_standardized_data(corpus_name, path, metadata):
    if corpus_name == 'FakeBrAccent':
        labels_dict = parse_fakebr_labels(metadata)
        files = get_audio_files(path)
        # Retornar lista de dicionarios padronizada
        dataset = []
        for f in files:
            base_name = os.path.basename(f).split('.')[0]
            dataset.append({
                'path': f,
                'label': labels_dict.get(base_name, 'unknown'),
                'corpus': corpus_name
            })
        return dataset
```

Capítulo 3

Medir custo em CPU

Medir o custo não é só sobre olhar para o relógio ao rodar o script. Precisamos medir funções específicas: I/O (leitura de disco), WPT (processamento matemático), cálculo de energia (operações estatísticas), treinamento e inferência.

3.1 Usando `time.time()`

A métrica mais básica é a diferença de tempo usando `time.time()`.

```
import time

def measure_basic_time():
    start = time.time()
    # Simula processamento
    time.sleep(0.5)
    end = time.time()
    return end - start

elapsed = measure_basic_time()
print(f"Tempo gasto: {elapsed:.4f} segundos")
```

3.2 Medindo Leitura vs. Decomposição

Nossa maior preocupação é saber o que demora mais: ler o áudio do disco ou decompor usando WPT?

```
import librosa
import pywt

def measure_read_vs_wpt(audio_path, wavelet='db4', level=3):
    # Medir leitura
    t0 = time.time()
    y, sr = librosa.load(audio_path, sr=None)
    t_read = time.time() - t0

    # Medir WPT
    t0 = time.time()
```

```
    wp = pywt.WaveletPacket(data=y, wavelet=wavelet, mode='
        symmetric', maxlevel=level)
    nodes = wp.get_level(level, 'freq')
    t_wpt = time.time() - t0

    return t_read, t_wpt
```

```
# Exemplo de execucao (simulado)
# t_r, t_w = measure_read_vs_wpt("exemplo.wav")
# print(f"Leitura: {t_r:.4f}s | WPT: {t_w:.4f}s")
```

3.3 Acompanhamento de Memória (tracemalloc)

As árvores WPT podem crescer rapidamente, ocupando muita RAM. Usaremos `tracemalloc` para aferir o pico de consumo de memória.

```
import tracemalloc

def measure_memory_wpt(audio_path, wavelet='db4', level=5):
    tracemalloc.start()

    y, sr = librosa.load(audio_path, sr=None)
    wp = pywt.WaveletPacket(data=y, wavelet=wavelet, mode='
        symmetric', maxlevel=level)
    nodes = wp.get_level(level, 'freq')

    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()

    # Retorna o pico de memoria em Megabytes
    return peak / (1024 * 1024)
```

```
# mem_peak = measure_memory_wpt("exemplo.wav", level=7)
# print(f"Pico de memoria (nivel 7): {mem_peak:.2f} MB")
```

Capítulo 4

Profiling detalhado

O módulo `cProfile` ou ferramentas integradas são úteis para descobrir gargalos dentro do código. Mas para os nossos relatórios, precisamos de médias.

4.1 Média de Múltiplas Execuções

Executar uma vez pode sofrer interferência do Sistema Operacional. Devemos fazer uma média com múltiplas rodadas.

```
import numpy as np

def benchmark_wpt(audio_path, runs=10):
    times = []
    y, _ = librosa.load(audio_path, sr=None)
    for _ in range(runs):
        start = time.time()
        pywt.WaveletPacket(data=y, wavelet='db4', mode='symmetric',
            maxlevel=5)
        end = time.time()
        times.append(end - start)

    return np.mean(times), np.std(times)
```

```
# mean_t, std_t = benchmark_wpt("exemplo.wav")
# print(f"Media de tempo: {mean_t:.4f}s +/- {std_t:.4f}s")
```

4.2 Custo de Inferência do Modelo

Além do WPT, precisamos saber quanto custa colocar um áudio no modelo (Ex: SVM ou Random Forest) *depois* de treinado.

```
def measure_inference_time(model, features_array):
    # Medir tempo de classificacao de um pacote de dados (batch)
    start = time.time()
    predictions = model.predict(features_array)
    end = time.time()

```

```
    total_time = end - start
    per_sample_time = total_time / len(features_array)

    return total_time, per_sample_time
```

```
# features_dummy = np.random.rand(100, 32) # 100 amostras, 32 features
# t_tot, t_ind = measure_inference_time(modelo_svm, features_dummy)
# print(f"Inferindo 100 amostras: {t_tot:.4f}s (Media: {t_ind:.5f}s/amostra)")
```

Capítulo 5

Tabela de Custos

Todo este esforço de coleta de tempos deve resultar numa tabela bonita usando Pandas.

```
import pandas as pd

def create_cost_dataframe(results_list):
    # results_list e uma lista de dicionarios
    df = pd.DataFrame(results_list)
    # Reordenando colunas para padronizacao
    cols = ['corpus', 'wavelet', 'nivel', 'tempo_read_ms',
            'tempo_wpt_ms', 'mem_pico_mb', 'tempo_treino_s', '
               tempo_inferencia_ms']

    # Filtra as colunas caso existam
    present_cols = [c for c in cols if c in df.columns]
    return df[present_cols]
```

```
dummy_results = [
    {'corpus': 'FakeBr', 'wavelet': 'db2', 'nivel': 4, '
       tempo_wpt_ms': 15.2, 'mem_pico_mb': 12.5},
    {'corpus': 'FakeBr', 'wavelet': 'db4', 'nivel': 6, '
       tempo_wpt_ms': 45.1, 'mem_pico_mb': 35.8},
]

df_cost = create_cost_dataframe(dummy_results)
# print(df_cost)
```

```
# Salvando tabela para o relatorio
def export_latex_table(df, filename):
    with open(filename, 'w') as f:
        f.write(df.to_latex(index=False, caption='Tabela de Custos
           (CPU/Memoria)', label='tab:custos'))

# export_latex_table(df_cost, 'custos.tex')
```

Capítulo 6

Ajuste de escopo

Se um teste no ASVspoof com Symlet8 nível 8 demorar 2 minutos por áudio na extração, processar 50.000 áudios levará muito tempo. Precisamos de lógica para cortar experiências caras.

```
def filter_viable_configurations(configs, max_time_per_audio_ms=100.0, max_mem_mb=100.0):
    viable = []
    for conf in configs:
        if conf['tempo_wpt_ms'] <= max_time_per_audio_ms and conf['mem_pico_mb'] <= max_mem_mb:
            viable.append(conf)
    return viable
```

```
# configs_teste = [{'name':'conf1', 'tempo_wpt_ms': 50, 'mem_pico_mb': 80},
#                  {'name':'conf2', 'tempo_wpt_ms': 250, 'mem_pico_mb': 150}]
# aceitaveis = filter_viable_configurations(configs_teste)
# print(aceitaveis) # Apenas conf1 sera mostrada
```

Capítulo 7

Pipeline de processamento em lote

Agora que sabemos processar e medir, vamos unificar com barra de progresso (`tqdm`), relatório de erros e *logging*.

```python
import logging
from tqdm import tqdm

logging.basicConfig(filename='processamento_mes08.log', level=
    logging.INFO)

def process_batch(audio_files, config):
    results = []
    for fpath in tqdm(audio_files, desc=f"Processando {config['
        corpus']}"):
        try:
            t0 = time.time()
            y, sr = librosa.load(fpath, sr=None)
            wp = pywt.WaveletPacket(y, wavelet=config['wavelet'],
                maxlevel=config['level'])
            # Calculo simples de feature (ex: energia por no)
            nodes = wp.get_level(config['level'], 'freq')
            feat = [np.sum(n.data**2) for n in nodes]
            tf = time.time() - t0

            results.append({'path': fpath, 'tempo_total': tf, '
                status': 'OK'})
        except Exception as e:
            logging.error(f"Erro no arquivo {fpath}: {str(e)}")
            results.append({'path': fpath, 'status': 'ERRO'})
    return results
```

```python
# Executando o pipeline (simulacao)
# batch_res = process_batch(['a.wav', 'b.wav'], {'corpus': 'ASV', '
    wavelet': 'db4', 'level': 3})
# ok_count = sum(1 for r in batch_res if r['status'] == 'OK')
# print(f"Sucessos: {ok_count}")
```

Capítulo 8

Brincadeiras: Otimizando o Tempo

Vamos realizar algumas brincadeiras investigativas usando NumPy vs Loops nativos.

8.1 Brincadeira 1: Soma de Energia com For Loop vs NumPy

```
def energia_loop(data):
    soma = 0
    for val in data:
        soma += val * val
    return soma
```

```
def energia_numpy(data):
    # Muito mais rapido devido ao C-backend
    return np.sum(np.square(data))
```

8.2 Brincadeira 2: `timeit`

```
import timeit

# Usaremos arrays do numpy para simular os dados do WPT
test_data = np.random.randn(100000)

t_loop = timeit.timeit(lambda: energia_loop(test_data), number=100)
t_np = timeit.timeit(lambda: energia_numpy(test_data), number=100)

print(f"Loop for: {t_loop:.5f}s")
print(f"NumPy: {t_np:.5f}s (Custo reduzido!)")
```

8.3 Brincadeira 3: Profiling com cProfile

Você pode exportar os resultados e usar ferramentas gráficas.

```
import cProfile

def rotina_pesada():
    arr = np.random.rand(500, 500)
    _ = np.linalg.inv(arr)

cProfile.run('rotina_pesada()')
```

8.4 Brincadeira 4: Garbage Collection

As vezes, os dados da memória RAM demoram para sumir após carregar vários *corpora*.

```
import gc

def limpar_memoria():
    # Coleta lixo manual, bom para o fim de processamentos pesados
    coletados = gc.collect()
    print(f"Objetos coletados: {coletados}")

# limpar_memoria()
```

Capítulo 9

Checklist

Criado script central de conversão de labels para os novos corpora.

Criada a função que unifica as rotas de áudio num dataset padrão.

Implementadas as funções `time.time()` em volta das funções de extração.

`tracemalloc` incluído para testes e decisões de corte por memória.

Critério de corte (*Scope adjustment*) definido (Ex: Max 500ms / 200MB por áudio).

Pipeline protegido com blocos `try/except` e barra de progresso.

Tabelas formatadas sendo exportadas em '.tex' ou CSV para o relatório.

Capítulo 10

Desafio final

Desafio

Utilizando as funções propostas, crie um script que faça o seguinte:

1. Escolha 100 arquivos aleatórios do *ASVspoof* e 100 do *FakeBrAccent*.
2. Calcule os *features* usando Symlet 5 (Nível 5) e colete o tempo WPT e de leitura.
3. Imprima uma estatística comparativa: O corpus influencia no tempo médio (talvez devido à duração média dos áudios)?

Dica: Ao comparar corpora, observe a taxa de amostragem original e o tamanho médio dos vetores resultantes.